

Building Synthetic Market Data Generators

ORE Studio Market Data Series — Part 1

ORE Studio

Agenda



The Underlying and the Problem of Market Data

A Trivial Stochastic Process: The Uniform Random Walk

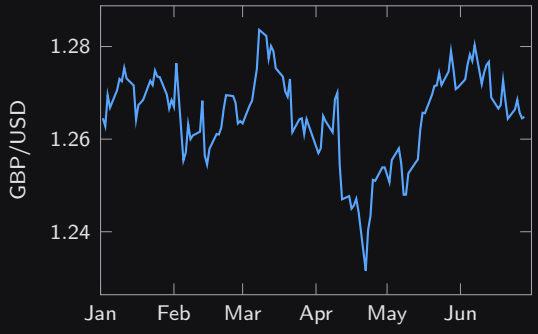
Measurements, Deviations and Log Returns

Discrete vs. Continuous Time and Space

Normal Distributions, Wiener Processes and Brownian Motion

The Underlying and the Problem of Market Data

Observing Reality: 6 Months of GBP/USD "Cable"



GBP/USD, Jan–Jun 2024. Daily ECB reference rates via Frankfurter

Before any code or models, look at real financial data.

What we notice:

- **Continuous and bounded:** prices move in smooth, connected trajectories
- **Memory:** today's price level is heavily constrained by yesterday's
- **Regimes:** low volatility alternates with high volatility

Underneath the Chart: The Raw Data Table



Every chart is a visual representation of a time-indexed table.

Date (t_i)	GBP/USD Spot (S_i)	Description
2024-01-02	1.2645	Starting baseline (S_0)
2024-01-03	1.2628	Small downward tick
2024-01-04	1.2695	Rebound
...	...	*126 rows of daily ECB reference rates*
2024-06-28	1.2648	Ending price (S_N)

Defining the Time Series: Software Mapping



To a software engineer, this data is a structured array.

Math	Software	Meaning in code
t_i	`timestamps[i]`	Ordered discrete time steps (e.g. daily ECB fixings)
S_i	`prices[i]`	The spot price stored at array index `i`
S_0	`prices[0]`	The starting price of the series
N	`len(prices)`	Total observation count (e.g. 126 trading days)

Why Do We Need Synthetic Data?



The ORE Studio challenge

- Risk engines must test thousands of *hypothetical futures* (Monte Carlo scenarios) to calculate potential future exposure and risk metrics
- Real historical data for every possible future scenario is restricted, expensive or non-existent

Our mission

Generate realistic artificial time series that mimic the statistical properties of the real GBP/USD chart.

A Trivial Stochastic Process: The Uniform Random Walk

What is a Stochastic Process?



The intuition

- A deterministic process always gives the same output for a given input (e.g. $y = 2x$)
- A stochastic process is a sequence of random variables indexed over time

The software engineer's definition

A loop over an array. Each step appends a value from a random number generator:

$$\text{prices}[i] = \text{prices}[i-1] + \text{RNG}()$$

Sampling from a Uniform Distribution



The simplest RNG: in $U(a, b)$, every value between a and b has equal probability.

Our first trivial model

- Daily moves $\Delta S_i = S_i - S_{i-1}$ come from a uniform distribution:

$$\Delta S_i \sim U(-0.0081, +0.0081)$$

- State update:

$$S_i = S_{i-1} + \Delta S_i$$

Code: Uniform Random Walk

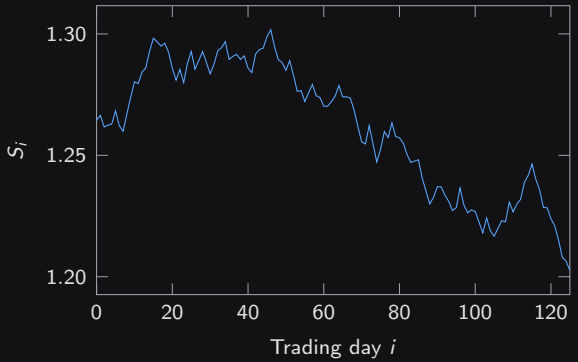


```python
import numpy as np
import pandas as pd

np.random.seed(42)
N = 126              # 6 months (~126 trading days)
S0 = 1.2645          # Initial GBP/USD spot price
step_bound = 0.0081 # Uniform bounds [-0.0081, +0.0081]

delta_S = np.random.uniform(low=-step_bound, high=step_bound, size=N - 1)
prices = np.zeros(N)
prices[0] = S0
prices[1:] = S0 + np.cumsum(delta_S)
```

Why the Eye Test Is Not Enough



One uniform walk: $N = 126$, $S_0 = 1.2645$, drawn by LaTeX at build time, not NumPy's seed 42

Visual observation

With its volatility matched to GBP/USD, the path looks plausible. Real and synthetic are hard to tell apart by eye.

The engineering problem

The model still has hard step bounds and no fat tails. "Looks right" cannot be a software assertion. We need numeric, quantitative metrics.

Measurements, Deviations and Log Returns

The Flaw of Raw Price Differences



Raw differences $S_t - S_{t-1}$ measure absolute price movement.

Move	Asset price	Relative size
\$1.00	\$10.00	10% (massive)
\$1.00	\$10,000	0.01% (trivial)

The flaw: raw deviations depend entirely on the absolute price level.

Measuring Spread: The Cancellation Problem



To see how spread out the data is, start with the distance of each return from the mean: $(r_i - \mu)$.

The mean is the exact centre. Positive and negative deviations cancel exactly to zero:

$$\sum_{i=1}^{N}(r_i - \mu) = 0$$

Fixing Sign Cancellation: Why Standard Deviation?

Option A: absolute deviations $|r_i - \mu|$
Intuitive, but $|x|$ is not differentiable at $x = 0$. This breaks calculus and optimisation.

Option B: squared deviations $(r_i - \mu)^2$
Removes negative signs, is smooth and differentiable, and heavily penalises extreme shocks.

Square root: restores the units of the data.

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(r_i - \mu)^2}$$



Moving to Returns: Prices are Multiplicative



From 1.20 to 1.26, GBP/USD did not add 0.06 like a counter. It scaled up by 5%: $1.20 \times 1.05 = 1.26$.

Step 1: simple percentage returns

$$R_t = \frac{S_t - S_{t-1}}{S_{t-1}} = \frac{S_t}{S_{t-1}} - 1$$

Limitations in software

- **Asymmetry trap:** a 50% drop ($100 \to 50$) needs a +100% gain ($50 \to 100$) to recover. This breaks symmetric assumptions.
- **Multiplicative chains:** compounding requires multiplication loops, which bring numerical drift and awkward window calculations:

$$(1 + R_{\text{total}}) = (1 + R_1) \times (1 + R_2) \times \cdots \times (1 + R_N)$$

Step 2: Why Log Returns Solve Engineering Problems



Logarithms convert multiplication into addition: $\ln(A \times B) = \ln(A) + \ln(B)$.

$$r_t = \ln\left(\frac{S_t}{S_{t-1}}\right) = \ln(S_t) - \ln(S_{t-1})$$

1. **Time additivity:** the total return over N steps is a vector sum, $r_{\text{total}} = \sum_{i=1}^{N} r_i$ (`np.sum()`)
2. **Symmetry around zero:** drops and gains are symmetric in log space, a clean fit for Gaussian distributions
3. **Non-negativity guarantee:** reconstruction uses exponentiation, $S_t = S_0 e^{\sum r_i}$, so prices never fall below zero. The Euler–Maruyama scheme later in this deck has no such guarantee.

Code: Measuring and Reconstructing Prices



```python
import numpy as np
# Continues the walk code: uses its prices array
# 1. Compute log returns array from price array: ln(S_t / S_{t-1})
log_returns = np.log(prices[1:] / prices[:-1])

# 2. Compute basic statistical parameters
mean_return = np.mean(log_returns)
volatility = np.std(log_returns, ddof=1)
print(f"Mean Daily Return (mu): {mean_return:.6f}")
print(f"Daily Volatility (sigma): {volatility:.6f}")

# 3. Reconstruct prices array from log returns via cumsum + exp
S0 = 1.2645
reconstructed_prices = S0 * np.exp(np.cumsum(np.insert(log_returns, 0, 0)))
```

Discrete vs. Continuous Time and Space

The Developer's Dilemma: Math vs. CPU



The math textbook (continuous world)

Stochastic calculus defines S_t and W_t for *every* real $t \geq 0$. You can zoom in infinitely between $t = 1.0$ and $t = 1.0001$.

The CPU and RAM (discrete world)

Memory consists of finite arrays (`std::vector<double>` or NumPy arrays) indexed by integers $i = 0, 1, 2, \ldots, N$.

The bridge: we **discretize** the equations and slice continuous time into finite steps Δt.

Mental Model: Video Game Frames vs. Analog Motion



Concept	Continuous reality (math)	Discrete implementation (code)
Analogy	A car on a highway, covering infinite points on the road	A video game at 60 FPS, updating the position every $\frac{1}{60}$ s
Time variable	$t \in \mathbb{R}$	Array index `i` (`t = i * dt`)
State evaluation	Can evaluate $S(t)$ at $t = 1.23456789\ldots$	Can only look up `prices[i]` at exact indices
Random shocks	Brownian motion W_t (unbroken random path)	A draw $Z \sim \mathcal{N}(0,1)$ at each discrete tick

The core rule

We use continuous-time calculus to derive the rules of motion. We always implement them as discrete recurrence relations in a `for` loop.

Translating Continuous Equations into Discrete Code



Continuous form (stochastic differential equation)

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

dt is an infinitesimally small time step. dW_t is a continuous Wiener increment.

Discrete approximation (Euler–Maruyama)

Replace dt with Δt (e.g. $1/252$ for a daily step). Replace dW_t with a normal draw scaled by $\sqrt{\Delta t}$:

$$S_i = S_{i-1} + \mu S_{i-1}\Delta t + \sigma S_{i-1}\sqrt{\Delta t}\, Z_i, \quad Z_i \sim \mathcal{N}(0,1)$$

Code: Discretization Parameters



```
import numpy as np

T = 0.5              # Total time horizon (0.5 years ~ 6 months)
N = 126              # Number of discrete steps (trading days)
dt = T / N           # Size of each discrete time step (Delta t)
S0 = 1.2645          # Initial spot price
mu = 0.0005          # Annual drift
sigma = 0.059        # Annual volatility

prices = np.zeros(N)
prices[0] = S0
```

Code: The Euler–Maruyama Update Loop



```
for i in range(1, N):
    Z = np.random.normal(0.0, 1.0)
    # Euler-Maruyama discrete update step
    dS = mu * prices[i-1] * dt + sigma * prices[i-1] * np.sqrt(dt) * Z
    prices[i] = prices[i-1] + dS

print("First 5 discrete price points:", prices[:5])
```

Normal Distributions, Wiener Processes and Brownian Motion



What real data shows

A histogram of real GBP/USD log returns forms a bell curve. Small changes happen constantly. Extreme changes happen rarely.

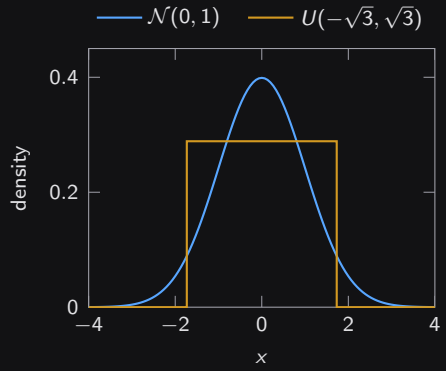
Gaussian definition

Defined by mean μ and variance σ^2:

$$r_t \sim \mathcal{N}(\mu, \sigma^2)$$

Gaussian vs. Uniform: Why It Matters

	$U(a, b)$	$\mathcal{N}(\mu, \sigma^2)$
Bounds	Hard cutoffs: impossible outside $[a, b]$	Infinite tails: any value is possible, though rare
Clustering	Every value in the range is equally likely	Values cluster heavily around μ



Both have variance 1

Code: Upgrading to a Normal Random Walk



```python
import numpy as np

mu = 0.000002  # Daily drift
sigma = 0.0037 # Daily volatility
N = 126

# 1. Draw standard normal random variables
Z = np.random.normal(loc=0.0, scale=1.0, size=N - 1)

# 2. Scale by volatility and add drift
log_returns = mu + sigma * Z

# 3. Reconstruct prices: S_t = S_{0} * exp(cumsum(log_returns))
prices = np.zeros(N)
prices[0] = 1.2645
prices[1:] = prices[0] * np.exp(np.cumsum(log_returns))
```



The Wiener process W_t: the continuous-time foundation of quantitative finance.

Over an interval Δt, the change ΔW is normal with mean 0 and variance Δt:

$$\Delta W = \sqrt{\Delta t} \cdot Z \sim \mathcal{N}(0, \Delta t), \quad Z \sim \mathcal{N}(0, 1)$$

The bridge to Part 2

The $\sqrt{\Delta t}$ scaling rule makes volatility scale correctly across daily, hourly or sub-second ticks. This sets up stochastic differential equations such as Geometric Brownian Motion.